Solució de l'Examen Parcial EDA 08/04/2026

Proposta de solució al problema 1

(a) Per a comparar les dues funcions, hem de calcular el límit

$$\lim_{n\to\infty} \frac{3^{n/2}}{2^n}$$

Podem reescriure

$$\lim_{n\to\infty} \frac{3^{n/2}}{2^n} = \lim_{n\to\infty} \frac{(3^{1/2})^n}{2^n} = \lim_{n\to\infty} \frac{(\sqrt{3})^n}{2^n} = \lim_{n\to\infty} \left(\frac{\sqrt{3}}{2}\right)^n$$

Com que $\sqrt{3} < 2$, el nombre que elevem a n és menor que 1 i sabem que aquests tipus de límits tendeixen a 0.

Així doncs, sabem que 2^n és major asimptòticament. És a dir, $3^{n/2} \in O(2^n)$.

(b)

a	m	k	$Cost$
2	$n/2$	1	$n \log n$
1	$n/2$	0	$\log n$
2	$n/2$	0	n
1	$n-1$	1	n^2
2	$n-1$	0	2^n

(c) Sigui x l'element del node buit. Com que està a la dreta del 8, necessàriament $x > 8$. D'altra banda, el seu fill esquerre és 12, pel que $x > 12$. Finalment, ha de ser menor que tots els nodes del seu subarbre dret, pel que $x < 15$. Resumint, tenim $12 < x < 15$, pel que els enters possibles són 13 i 14.

Proposta de solució al problema 2

(a) Una possible forma de completar el codi:

```
bool search1(int x, const vector<vector<int>>& A, int i, int j, int n) {
if (n == 1) return A[i][j] == x;
int mi = i + n/2 - 1;
int mj = j + n/2 - 1;

if (A[mi][mj] < x) return
    search1(x, A, mi+1,    j,    n/2) or
    search1(x, A, mi+1, mj+1, n/2) or
    search1(x, A,    i,  mj+1, n/2);

if (A[mi][mj] > x) return
    search1(x, A, mi+1,    j,    n/2) or
    search1(x, A,    i,    j,    n/2) or
    search1(x, A,    i,  mj+1, n/2);

return true;
}
```

Observem que el resultat de cridar *search1*$(x,\ A,\ 0,\ 0,\ N)$ és **true** si i només si x apareix a A.

Per analitzar el cost d'aquesta crida, en primer lloc estudiem el cas general de cridar *search1*$(x,\ A,\ i,\ j,\ n)$ en funció de n. Sigui $T(n)$ el cost en el cas pitjor d'aquesta crida. Com que en el cas pitjor es fan 3 crides amb darrer paràmetre $n/2$ i el cost del treball no recursiu és constant, tenim que se segueix la recurrència:

$$T(n) = 3T(n/2) + \Theta(1)$$

Aplicant el teorema mestre de recurrències divisores, tenim $T(n) = \Theta(n^{\log_2 3})$.

Així doncs, el cost de cridar *search1*$(x,\ A,\ 0,\ 0,\ N)$ és $\Theta(N^{\log_2 3})$.

(b) El bucle manté el següent invariant: si x apareix a A, llavors ho fa entre la fila 0 i la i (incloses), i entre la columna j i la $N-1$ (incloses). En començar el bucle, l'invariant és cert. I a cada volta es manté:

- Si $A[i][j] > x$ aleshores x no pot aparèixer a la fila i: de l'invariant tenim que si x apareix ha de ser en una columna de la j a la $N-1$. Però si $j \leq k < N$ llavors $A[i][k] \geq A[i][j] > x$.
- Si $A[i][j] < x$ aleshores x no pot aparèixer a la columna j: de l'invariant tenim que si x apareix ha de ser en una fila de la 0 a la i. Però si $0 \leq k \leq i$ llavors $A[k][j] \leq A[i][j] < x$.

Per últim, si el programa retorna **true** pel **return** de dins del bucle, la resposta és correcta perquè $A[i][j] = x$. Si retorna **false** pel **return** de fora, llavors $i < 0$ o $j \geq N$. En qualsevol cas, de l'invariant deduïm que x no apareix a A.

(c) A cada volta o bé i es decrementa en 1, o bé j s'incrementa en 1, o bé es retorna. A més, inicialment i val $N-1$, i com a mínim val 0 abans de sortir del bucle. Similarment, al principi j val 0 i com a molt val $N-1$ abans de sortir del bucle. Com que en el cas pitjor es poden fer $2N-1$ voltes i cadascuna té cost $\Theta(1)$, el cost és $\Theta(N)$.